FOUNDATIONS OF WEB APP PENTESTING



Section 1.1

Introduction

High-Level Overview • How Web Applications Work

HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

1.1 SECTION INTRODUCTION

HIGH-LEVEL OVERVIEW

HOW WEB APPLICATIONS WORK

FIRST UNDERSTAND TWO WORDS

WEB

"Web" simply means **the internet**.

Example:

- Google
- Facebook
- YouTube

All of these run on the **web (internet)**.

APPLICATION

An **application** means **a program that does something useful**.

Examples of applications:

Application	What it does
Calculator	Calculates numbers
WhatsApp	Sends messages
YouTube	Plays videos

So:

Web Application = A program that runs on the internet.

Example web apps:

- Gmail
- Facebook

- Online banking
- Amazon
- Twitter

These are **not simple websites** — they are **interactive systems**.

SIMPLE WEBSITE vs WEB APPLICATION

SIMPLE WEBSITE

Example: A blog page.

You open it and just **read information**.

You do **not interact much**.

Example actions:

- Read article
- View images

That is basically it.

WEB APPLICATION

A web application **interacts with you**.

Example: Gmail

You can:

- Login
- Send emails
- Delete emails
- Upload attachments
- Search emails

This requires **many systems working together**.

WHAT IS A WEB APPLICATION?

A **web application** is a program that runs on the internet and that you use through a **web browser**.

Examples of web applications:

- Gmail
- Facebook
- Amazon

- Online banking
- YouTube

You do not install them on your computer like normal software.

Instead:

1. You open a **browser** (Chrome, Firefox, Edge)
2. You visit a **website**
3. The application runs through the internet

Example:

When you open **YouTube** in Chrome and watch a video, you are using a **web application**.

WEB APPLICATION = CLIENT + SERVER

A web application always works using two main sides.

```
USER → CLIENT (BROWSER) → INTERNET → SERVER → DATABASE
```

Think of it like ordering food in a restaurant.

Real World	Web Application
Customer	Client
Waiter	Internet
Kitchen	Server
Storage room	Database

Flow example:

1. You ask for food
2. Waiter takes request to kitchen
3. Kitchen prepares food
4. Food comes back to you

Same happens in web apps.

THE THREE CORE COMPONENTS

Every web application mainly has **three parts**:

- Client (Front-End)
- Server (Back-End)

- Database

We will understand each one.

1. THE CLIENT (FRONT-END)

What it means

The **client** is the user's environment **(device + browser)** where the application runs.

The **front-end** is the part of the application that **you see and interact with**.

This runs inside your **web browser**.

Example things you see:

- Login page
- Buttons
- Forms
- Images
- Videos
- Text

All of this is called the **Front-End**.

Technologies used in the Client

Three main technologies build the front-end.

I. HTML

HTML builds the **structure** of the webpage.

Example:

- `Login Page`
- `Username box`
- `Password box`
- `Login button`

Think of HTML as the **skeleton of a webpage**.

II. CSS

CSS controls **how things look**.

Examples:

- colors
- fonts
- spacing

- layout

Example:

Without CSS → ugly page

With CSS → beautiful design

Think of CSS as **makeup and clothing for the webpage**.

III. JavaScript

JavaScript adds **behavior and interaction**.

Example:

- Button clicks
- Form validation
- Popups
- Live updates

Example:

If you enter wrong password → message appears

That behavior usually comes from **JavaScript**.

Simple Example

When you open a **login page**:

You see:

- `Username box`
- `Password box`
- `Login button`

That is the **front-end** running in the **client**.

But the browser **does not securely verify if the password is correct**.

Instead it sends the request to the **server**.

PENTESTER FOCUS (CLIENT SIDE)

A penetration tester checks if the client side can be abused.

Because **everything in the browser is visible and controllable by attackers**.

Two common issues exist

I. Cross-Site Scripting (XSS)

XSS happens when attackers inject **malicious JavaScript** into a website.

Example attack:

A comment box on a website.

Normal user writes: `Great article!`

Attacker writes: `<script>alert("Hacked")</script>`

If the website does not filter this properly, the script runs in other users' browsers.

That can allow attackers to:

- steal cookies
- hijack sessions
- run malicious code

II. Sensitive Information in Source Code

Sometimes developers accidentally leave important information in the **frontend code**.

Example:

Inside JavaScript:

```
API_KEY = "123456SECRET"
```

Or:

```
admin_password = "admin123"
```

Anyone can open **View Page Source** and see this.

That is a security risk.

2. THE SERVER (BACK-END)

What it means

The server is the machine that runs the application.

The back-end is the logic (code) running on that server.

The server:

- receives requests
- processes them (through back-end logic)
- sends responses back

You cannot see the server directly.

It runs somewhere on the internet.

Example

You log into a website.

Steps:

1. Browser sends username and password
2. Server receives the request
3. Back-end checks database
4. Server sends result

Possible responses:

`Login successful` or `Invalid password`

What the server does

The back-end handles **business logic**.

Business logic means **rules of the application**.

Example rules:

Bank website:

```
User must be logged in to transfer money
User must have enough balance
```

Shopping website:

```
Item must exist
Payment must be valid
```

All these checks happen **in the back-end on the server**.

Common Web Servers

Some popular web servers include:

- Apache
- Nginx
- Microsoft IIS

These programs help:

- receive web requests
- deliver web pages
- run backend applications

Think of them as **traffic controllers for web requests**.

PENTESTER FOCUS (SERVER SIDE)

Security testers look for problems on the server and back-end.

Common issues include

I. Server Misconfiguration

Sometimes servers are configured badly.

Examples:

Directory listing enabled.

You open:

```
example.com/uploads/
```

And see:

```
file1.zip
database_backup.sql
passwords.txt
```

That should **never happen**.

II. Outdated Software

Old server software often contains **known vulnerabilities**.

Example:

Old Apache version → known exploit exists.

Attackers can abuse it to:

- run commands
- access files
- take control of server

III. Bypassing Server Validation

Developers sometimes only check things on the **client side**.

Example:

Website allows only **numbers** in a form.

Client JavaScript checks this.

But attacker can send request directly:

```
POST /transfer
amount=1000<script>
```

If server does not validate input → vulnerability exists.

3. THE DATABASE

What it means

The **database** is where the application stores information.

Examples of stored data:

- usernames
- passwords
- emails
- orders
- messages
- blog posts

Without a database, websites could not remember anything.

Example

When you create an account:

```
Username: Haroon
Password: 123456
Email: haroon@email.com
```

The server stores this inside the **database**.

Later when you login:

The server checks the database to verify credentials.

Common Databases

Popular databases include:

- MySQL
- PostgreSQL
- MongoDB
- Microsoft SQL Server

These systems store large amounts of structured (or semi-structured) data.

PENTESTER FOCUS (DATABASE)

Security testers check if attackers can access the database improperly.

One famous vulnerability is

I. SQL Injection (SQLi)

SQL Injection happens when attackers manipulate database queries.

Example login query:

```
SELECT * FROM users WHERE username='haroon' AND password='123456'
```

Attacker enters:

```
Username: admin
Password: ' OR '1'='1
```

Query becomes:

```
SELECT * FROM users WHERE username='admin' AND password='' OR '1'='1'
```

Since `'1'='1'` is always true, login may succeed.

Attackers could then:

- login as admin
- dump user data
- modify database

COMPLETE WEB APPLICATION FLOW (SIMPLE EXAMPLE)

Let's see the **full process** when a user logs in.

Step 1 — User action

User enters:

```
username = haroon
password = 123456
```

Step 2 — Client sends request

Browser sends request to server.

```
POST /login
username=haroon
password=123456
```

Step 3 — Server processes request

Server receives request.

Back-end handles the processing.

Server checks: `Does this user exist?`

Step 4 — Server queries database

Server sends query: `SELECT * FROM users WHERE username='haroon'`

Database returns stored password.

Step 5 — Server compares password

Server checks: `Is password correct?`

Step 6 — Server responds

If correct: `Login successful`

If wrong: `Invalid credentials`

—----------

SIMPLE SUMMARY

A **web application** works using three main parts.

1. Client (Front-End)

Client = browser

Front-End = Code that runs in the browser.

Built with:

- HTML
- CSS
- JavaScript

User interacts with it.

Pentester checks:

- XSS
- exposed secrets

2. Server (Back-End)

Server = machine

Back-End = logic running on it

Processes requests and runs application logic.

Common servers:

- Apache
- Nginx
- IIS

Pentester checks:

- misconfigurations
- outdated software
- validation bypass

3. Database

Stores application data.

Examples:

- users
- passwords
- orders

Pentester checks:

- SQL Injection
- unauthorized access
